FITPATH

Project description, product requirements, technical architecture & implementation strategy

Course final project
Flutter Android application · Google Play target · Local-first MVP



Design reference supplied by the project owner. The calorie and macronutrient summary card is a preferred visual direction, not a finalized screen specification.

Working specification · 3 November 2026

Document overview

This document consolidates the FitPath concept and decisions made so far. It is intended to serve as a shared product and engineering baseline for implementation, testing, and the final course submission.

Item	Decision
Product	FitPath — personalized fitness, nutrition, and workout tracking
Target platform	Android, built with Flutter; intended for Google Play publication
MVP account model	No registration or login; one local user profile per app installation
Persistence	SQLite database through Drift; preferences stored separately
Connectivity	Core tracking works offline; external food search requires internet
Backup	User-initiated JSON export and import
Measurement units	User chooses metric or imperial during onboarding; canonical internal units
Food catalog	Hybrid: bundled/local foods, external food lookup, and custom foods
Workout scope	Log individual workout sessions; routine builder is out of MVP

Contents

1. Product vision and scope
2. Goals, users, and product principles
3. Onboarding and navigation
4. Screen and feature specifications
5. Dashboard and visual direction
6. Functional and non-functional requirements
7. Technical architecture
8. Database design
9. Food data and nutrition calculations
10. Workout data and progress calculations
11. Backup, privacy, and offline behavior
12. Implementation roadmap
13. Testing and acceptance criteria
14. Risks, assumptions, and open decisions

1. Product vision and scope

FitPath is a personalized fitness companion that helps users understand and manage their nutrition, record workouts, and monitor progress toward fitness goals. It combines calorie and macronutrient tracking, workout logging, body-weight history, and a customizable home dashboard.

The application is designed for people who want a practical way to track fitness-related information without needing to create an online account. Its dashboard adapts to the user's selected primary goal, while still allowing users to choose which information matters most to them.

Product proposition

A goal-aware, customizable fitness tracker that keeps the user's core data on their device and makes nutrition and training progress easy to understand.

MVP boundaries

- Android application implemented in Flutter and prepared for Google Play release.
- No user accounts, authentication, or required cloud backend.
- Local profile, goals, nutrition diary, workout history, and dashboard configuration.
- Basic calculations and progress visualizations.
- Food lookup using a hybrid catalog: local, external, and user-created foods.
- Individual workout logging; no workout routine builder in the MVP.
- User-controlled data export and import.

Out of scope for MVP

- Cloud synchronization, multi-device accounts, and social/community features.
- AI-generated workouts or recipes, image-based food recognition, and natural-language logging.
- Barcode scanning unless implementation time remains.
- Workout routine planning and execution.
- Automated medical advice or guaranteed weight-change predictions.

The application provides estimates and tracking tools, not medical diagnosis or individualized medical treatment. Calorie and weight-change estimates must be presented as approximations.

2. Goals, users, and product principles

Target users

The initial audience is students and general fitness enthusiasts who want to track food intake, workouts, and progress in one place. The MVP should be approachable for beginners while retaining useful detail for users who already understand sets, repetitions, weights, calories, and macronutrients.

Supported primary goals

Goal	Dashboard emphasis	Relevant setup information
Lose weight	Calories consumed/remaining, weight trend, nutrition	Current weight, optional target weight/date, activity level
Build muscle	Protein, calories, workouts, strength progression	Training experience, frequency, optional target weight
Maintain weight	Calorie balance, weight trend, activity	Current weight and activity level
Improve strength	Workout logging and exercise performance	Training experience and typical training frequency
General fitness	Workout consistency, nutrition, and overall progress	Activity level and preferred training frequency

Product principles

- **Local-first:** core user data remains available without an account or network connection.
- **Goal-aware, not goal-locked:** a preset dashboard is generated from the primary goal, but users can customize it.
- **Transparent estimates:** show that calorie and metabolic estimates are approximate and explain their inputs.
- **Historical integrity:** changes to a food catalog item or exercise name must not rewrite past logs.
- **Progressive scope:** deliver a dependable logging MVP before adding more complex recommendations.

Key product distinction

The user's fitness goal and dashboard layout are separate concepts. Changing the dashboard must not change the goal, and changing the goal should offer a new preset without silently deleting the user's historical data or custom layout.

3. Onboarding and navigation

First-launch flow

Step	Screen	Behavior
1	Welcome	Introduce FitPath and its value; provide Get Started action.
2	Let's get you set up	Collect age, height, current weight, activity level, unit preference, and optional sex-related metabolic input.
3	Choose your goals	Select one primary goal and optional secondary goals.
4	Configure goals	Ask only relevant follow-up questions, such as optional target weight/date or training frequency.
5	Initial estimates	Show estimated BMR/TDEE and suggested calorie/macronutrient targets, with explanations and editable values.
6	Create dashboard	Apply the goal-based preset, save the profile locally, and open Home.

Onboarding behavior and validation

- First launch is detected using a persisted onboarding-completed flag.
- Users can skip optional profile details; features requiring missing inputs should explain what is needed.
- Unit preference is selected during onboarding, with a sensible default preselected.
- Validate plausible ranges for age, height, weight, and dates; do not silently accept invalid values.
- Users can edit profile details and goals later without deleting workout or nutrition history.
- On subsequent launches, restore local state and open the main application directly.

Main navigation

Destination	Primary responsibility
Home	Goal-aware customizable dashboard and daily summary.
Nutrition	Food search, food diary, daily calorie and macro totals.
Workouts	Start, record, complete, and review individual workout sessions.
Progress	Historical weight, nutrition, and workout visualizations.
Profile & Settings	Personal details, goals, units, dashboard preferences, export/import, and app settings.

Use a persistent bottom navigation bar for the five main destinations. Detail screens should be pushed within their respective navigation flows, with a clear way to return.

4. Screen and feature specifications

Home dashboard

- Show a greeting, current date, and concise primary-goal context.
- Render the user's saved cards in their configured order.
- Provide an edit mode to add, hide, remove, and reorder cards.
- Persist dashboard changes automatically.
- Show helpful empty states when the user has not logged food, weight, or workouts yet.

Nutrition

Screen	Core behavior
Daily overview	Select a date; show calories and macros against targets; show meal groups and logged foods.
Food search	Search local foods first, then external catalog when appropriate and online.
Food details	Show serving basis, calories, protein, carbohydrates, fats, and source; choose quantity.
Add/edit food entry	Choose meal type and date/time; save; allow later edit or deletion.
Custom food	Create a food with a name, serving basis, and available nutritional values.

Workouts — logging only

- Start a new session and optionally provide a workout name.
- Add exercises from a predefined catalog or create a custom exercise.
- For each exercise, log sets, repetitions, and external load where applicable.
- Allow sets to be marked complete; allow a session to be completed or discarded.
- Review completed sessions and their exercise/set details.
- Do not include routine templates or a routine builder in the MVP.

Progress

- Weight history chart and change over selectable time ranges.
- Workout history and completed session counts.
- Nutrition history and daily calorie/macronutrient summaries.
- Exercise performance history based on logged sets.
- Clearly distinguish measured values from calculated estimates.

Profile & Settings

- Edit profile, activity level, goals, and nutrition targets.
- Change metric/imperial display preference without changing stored canonical values.
- Customize dashboard and theme/preferences as supported.
- Export data, import a backup, and reset local data with confirmation.

5. Dashboard and visual direction

The uploaded reference image establishes a preferred visual direction for the main calorie and macronutrient card. FitPath should use this as inspiration for the information hierarchy and visual treatment, rather than copying every detail literally.



User-provided visual reference: calorie goal, remaining calories, circular calorie visualization, and separate macronutrient progress indicators.

Preferred card content and hierarchy

- **Header:** daily calorie goal and remaining calories.
- **Main visualization:** circular or ring-style indicator showing calories consumed relative to the daily target.
- **Macro indicators:** separate progress bars for protein, fat, and carbohydrates, each showing consumed amount versus target.
- **Daily activity context:** optional compact row for meals logged or other relevant daily status.
- **Date selection:** allow users to inspect another day without confusing historical data with today's totals.

Design interpretation

The reference uses a warm off-white surface, dark serif-style headings, a warm orange accent for the selected date/highlight, and distinct colors for macro indicators. The design feels softer and more editorial than a typical high-contrast gym tracker. FitPath can use a related palette while maintaining readable text, accessible contrast, and consistent components.

Data behavior

- Calories remaining = daily calorie target minus logged calories. If consumption exceeds the target, show the over-target amount rather than a negative 'remaining' value.

- Macro progress should show consumed grams and target grams independently.
- If a macro target is unset, display an unconfigured state rather than a misleading zero target.
- The ring should handle zero targets and over-target values safely.
- The card is a dashboard component; the detailed Nutrition screen remains the source for food entry and editing.

Goal-based presets

Primary goal	Suggested initial cards
Lose weight	Calories/macros, weight trend, daily nutrition, workout summary
Build muscle	Calories/macros with protein emphasis, workout summary, strength progression, weight trend
Maintain weight	Calories/macros, weight trend, workout summary
Improve strength	Workout summary, strength progression, recent sessions, nutrition summary
General fitness	Workout summary, activity overview, nutrition summary, weight trend

6. Functional and non-functional requirements

Functional requirements

ID	Requirement
FR-01	The first launch must present welcome and onboarding before the main dashboard.
FR-02	The user can create and edit one local profile without registering.
FR-03	The user can select metric or imperial units.
FR-04	The user can select one primary goal and optional secondary goals.
FR-05	The app calculates and displays BMI, BMR, and TDEE estimates where sufficient inputs exist.
FR-06	The user can configure calorie and macronutrient targets.
FR-07	The user can search, select, create, edit, and log foods.
FR-08	The app stores nutrition snapshots for each food diary entry.
FR-09	The user can create and complete individual workout sessions.
FR-10	The user can log exercises, sets, repetitions, and weights.
FR-11	The user can record and view body-weight history.
FR-12	The user can add, hide, and reorder dashboard cards.
FR-13	The user can view historical progress summaries and charts.
FR-14	The user can export and restore local application data.

Non-functional requirements

ID	Requirement
NFR-01	Core profile, diary, workout, and progress data must remain usable offline.
NFR-02	External API failure must not prevent logging cached or custom foods.
NFR-03	Database writes involving related records must use transactions.
NFR-04	Database schema changes must use migrations that preserve existing user data.
NFR-05	The UI must provide loading, empty, validation, and error states.
NFR-06	Data export/import must validate format version and content before modifying the database.
NFR-07	The app must avoid unnecessary collection or transmission of personal data.
NFR-08	Text, charts, and controls must remain usable on common Android screen sizes and accessibility font settings.

7. Technical architecture

Layer	Responsibility	Suggested technology
Presentation	Screens, reusable widgets, navigation, UI state	Flutter, Riverpod, go_router
Domain	Business rules, calculations, use cases, domain models	Dart; framework-independent where practical
Data	Repositories, local persistence, API integration, serialization	Drift, SQLite, Dio, json_serializable
External data	Food product search and retrieval	Open Food Facts API
Visualization	Progress charts and dashboard indicators	fl_chart
Testing	Unit, widget, and integration tests	flutter_test; Drift test database

Recommended project structure

```
lib/
main.dart
app.dart
core/
database/ network/ errors/ utils/ theme/
features/
onboarding/
profile/
dashboard/
nutrition/
workouts/
progress/
settings/
shared/
widgets/ models/ extensions/
```

Within each substantial feature, separate presentation, domain, data, and providers/state management. Avoid creating empty abstractions prematurely; introduce files as the feature is implemented.

Dependency direction

Screens and widgets call providers/controllers. Those invoke domain use cases or repositories. Repositories coordinate Drift and external services. The domain calculation layer should not depend on Flutter widgets or the database implementation.

Example: logging a workout

Workout screen → workout state/provider → workout repository → Drift transaction → SQLite. Reactive providers then refresh workout history and dashboard summaries.

State management guidance

- Use Riverpod providers for reactive data and application state.
- Keep form-local state local to the screen when it does not need to be shared.
- Use repositories as the boundary for persistence and remote data.
- Keep calculations in pure Dart functions or services so they can be tested without Flutter UI.

8. Database design

Use SQLite through Drift. The app installation represents one local user, so user_id columns are unnecessary in every table. Keep a single profile row and associate records through their domain relationships.

Core tables

Table	Purpose and key fields
user_profile	One local profile: age, height_cm, optional sex input, activity_level, unit_system, created_at, updated_at.
fitness_goals	Goal history: goal_type, is_primary, target_weight_kg, optional target_date, is_active, created_at, completed_at.
weight_entries	Measurements: weight_kg, recorded_at, optional notes.
nutrition_targets	Target history: calories, protein_g, carbs_g, fat_g, effective_from, source.
food_items	Catalog: name, brand, nutrients, reference_basis, source, external_id, optional serving information.
food_entries	Diary: food_id, quantity, quantity_unit, nutrient snapshots, meal_type, consumed_at, local_date.
exercises	Exercise catalog: name, muscle_group, equipment, description, is_custom, optional deleted_at.
workout_sessions	Session: name, started_at, completed_at, notes, status.
workout_exercises	Exercise in a session: session_id, exercise_id, exercise_name_snapshot, order_index.
workout_sets	Set detail: workout_exercise_id, set_number, repetitions, nullable weight_kg, completed, notes.
dashboard_cards	Card layout: card_type, position, is_visible, card_size, optional configuration JSON.
app_settings	User preferences: theme and other lightweight preferences.
app_metadata	Technical state: schema/data format versions and onboarding completion.

Important modeling decisions

- **Current weight:** derive it from the most recent weight entry instead of storing a duplicate value in the profile.
- **Historical goals:** deactivate or complete old goals rather than overwriting them.
- **Nutrition snapshots:** store consumed nutrient values in food_entries so catalog edits do not rewrite history.
- **Reference basis:** food values must identify whether they are per 100 g, per 100 ml, or per serving. Do not assume grams and milliliters are interchangeable.
- **Exercise snapshots:** preserve the exercise name used in historical workout records.
- **Workout status:** distinguish in-progress, completed, and discarded sessions. Only completed sessions count toward completed workout summaries.
- **Dashboard configuration:** store one row per card so cards can be independently reordered and hidden.

Relationships

fitness_goals, weight_entries, food_entries, workout_sessions, and dashboard_cards are local-profile-owned records. food_entries reference food_items. workout_sessions contain workout_exercises, which reference exercises and contain workout_sets. Use foreign keys and enable SQLite foreign-key enforcement.

Deletion policy

- Deleting a workout session may cascade to its workout_exercises and workout_sets.

- Do not cascade-delete food diary entries when a food catalog item is removed; restrict deletion or mark the catalog item inactive.
- Do not destroy historical workout data when an exercise is renamed or removed; use snapshots and soft deletion where appropriate.

9. Food data and nutrition calculations

Hybrid food catalog

Use a bundled starter catalog for common foods, Open Food Facts for external packaged-food lookup, and custom foods for user-entered items. External results should be cached locally when selected so the user can log them later while offline.

Food search flow

```
1. Search the local catalog.
2. If local results are insufficient and internet is available, query the external service.
3. Display the source and available nutrition information.
4. When selected, cache the food in the local catalog.
5. Create a food diary entry with a snapshot of the nutritional values used.
```

Food data quality and licensing

Open Food Facts is community-contributed, so records may be incomplete or inaccurate. Show the source, handle missing nutrient values as unknown rather than zero, and review API terms, attribution requirements, and database licensing before publication. External API failure must not block local or custom-food logging.

Nutrition totals

For a food item with nutrients expressed per 100 g, a quantity q grams gives:

```
nutrient consumed = nutrient per 100 g × q / 100
```

For per-serving foods, use the serving amount and unit defined by the food record. Keep mass-based and volume-based references explicit.

Energy and metabolic estimates

- **BMI:** weight in kilograms divided by height in meters squared. Explain that BMI is a screening measure and does not describe body composition.
- **BMR:** use a documented estimation equation with its required inputs. If optional inputs are missing, explain the limitation or present an appropriately qualified estimate.
- **TDEE:** estimate from BMR and an activity factor; label it as an estimate.
- **Calorie target:** suggest a moderate, configurable target based on the user's goal. Avoid promising an exact weight change by a date.
- **Macros:** allow suggested targets to be edited; ensure the UI explains whether targets are calculated or user-adjusted.

Health and safety presentation

Do not frame calorie calculations as medical advice. Avoid extreme targets and unrealistic timelines. Use clear disclaimers, sensible validation, and encourage professional guidance where a user's circumstances require it.

10. Workout data and progress calculations

MVP workout flow

Start session → name session → add exercise → record sets/repetitions/load → add another exercise or finish → save completed session → review history.

Workout scope

- No account, social sharing, or routine builder is required.
- Provide a starter exercise catalog and allow custom exercises.
- Support bodyweight exercises and exercises where external load is not applicable.
- Allow a session to remain in progress if the app is closed, then resume or discard it.
- Use a database transaction when saving a completed session and its child records.

Progressive overload tracking

The MVP can show historical performance for the same exercise by comparing logged sets across completed sessions. A simple comparison can include load, repetitions, and total volume (load × repetitions) where those measures are meaningful.

Do not assume volume is comparable for every exercise type. Bodyweight, assisted, timed, distance-based, and unilateral exercises may need different measurement conventions in a later iteration.

Priority 2 data readiness

Feature	Data already captured
Monthly workout counts/streaks	Completed session timestamps and status
Personal records	Exercise identity and set-level repetitions/load
1RM estimates	Load and repetitions for suitable resistance exercises
What-if fitness calculator	Profile, goals, nutrition targets, and weight history
Weekly reports	Food entries, workout sessions, and weight history

11. Backup, privacy, and offline behavior

Local-first behavior

- Profile, goals, weight entries, food diary, cached/custom foods, workouts, and dashboard layout are stored locally.
- Food search against the external service requires internet, but previously cached and custom foods remain usable offline.
- A network failure should be reported clearly and should not erase or block locally available data.
- Do not require account creation, authentication, or a backend for MVP functionality.

JSON backup format

Export a versioned JSON document containing all user-owned records and necessary settings. Include an export format version and timestamp. Do not rely on database auto-increment IDs being globally unique; preserve relationships using exported IDs or stable identifiers within the backup.

Import/restore rules

- Validate the file structure, version, required fields, and data types before modifying the database.
- Show a confirmation that restore replaces current local data.
- Offer or recommend exporting the current data before replacement.
- Perform restore in a database transaction; roll back if any record fails validation or insertion.
- Do not import external API credentials or secrets.
- Test round-trip behavior: export → clear test database → import → compare records and relationships.

Privacy and Play Store considerations

Collect only information needed for the stated features. Explain that profile, weight, nutrition, and workout records are stored on the device. Review Google Play's current Data safety and privacy policy requirements, as well as any disclosures required by the external food API. Local-first does not automatically exempt an app from store disclosure obligations.

Data loss limitation

Uninstalling the app or clearing its storage may remove local records. Make the export feature discoverable and explain this limitation to users. Android device backup may be considered separately, but user-controlled export/import is the defined MVP backup approach.

12. Implementation roadmap

Implement in vertical slices so each milestone produces working functionality rather than building every layer in isolation.

Phase	Deliverable	Exit condition
1. Foundation	Flutter project, theme, navigation, Riverpod, Drift setup	App launches and navigates; database opens in tests.
2. Onboarding	Welcome, profile, unit selection, goals, setup completion	First-run flow saves data; later launches skip onboarding.
3. Domain calculations	BMI, BMR/TDEE, calorie and macro target services	Calculation unit tests and validated UI outputs.
4. Dashboard	Goal presets, card library, reorder/hide, calorie/macro card	Dashboard layout persists after restart.
5. Nutrition	Local catalog, API search, cache, custom foods, diary	Food can be logged offline from local/cache; totals are correct.
6. Workouts	Exercise catalog, sessions, exercise rows, sets, history	Completed session persists with all sets and appears in history.
7. Progress	Weight history, nutrition summaries, workout history, charts	Charts use persisted records and handle empty data.
8. Backup	Versioned JSON export/import	Round-trip and invalid-file tests pass.
9. Release readiness	Accessibility, error handling, performance, store assets	Release build tested on representative Android devices.

Priority 2

After the MVP is stable, implement the what-if fitness calculator, monthly workout counts/streaks, personal records, recipe recommendations, ingredient substitutions, 1RM estimates, and weekly reports. The current MVP data model is designed to support these without requiring them in the initial release.

Priority 3

Potential future work includes AI-assisted recipe/workout suggestions, natural-language food logging, barcode scanning, food image recognition, Health Connect integration, and optional cloud synchronization.

13. Testing and acceptance criteria

Testing strategy

Test level	Examples
Unit	Unit conversions; BMI/BMR/TDEE; calorie totals; macro totals; target calculations; date grouping.
Repository/database	CRUD operations; foreign keys; transactions; migrations; snapshots; deletion behavior.
Widget	Onboarding validation; food logging form; workout set entry; dashboard card editing; empty/error states.
Integration	First launch; restart persistence; offline food logging; external API failure; export/import round trip.
Release/device	Different screen sizes; Android back behavior; permissions; file picker/share flow; release build.

MVP acceptance criteria

- A first-time user can complete onboarding without creating an account.
- A returning user sees their previously saved profile and dashboard.
- Changing display units does not change the underlying stored measurements.
- The user can log food and see correct daily calorie/macronutrient totals.
- A food log remains historically stable if the catalog record is edited later.
- The user can log a workout with multiple exercises and sets, complete it, and review it later.
- The user can record body weight and see it reflected in progress history.
- Dashboard card visibility and order persist after restarting the app.
- The user can export and restore a backup without losing relationships between records.
- Core local functionality remains available when the device is offline.

Definition of done for a feature

A feature is complete when its main interaction works, data persists correctly, validation and error states exist, relevant tests pass, and the behavior is documented where it affects other features.

14. Risks, assumptions, and open decisions

Risk / decision	Mitigation or next action
Food data completeness	Handle missing nutrients, show source, allow custom foods, and test API failure.
External API terms	Review attribution, licensing, rate limits, and acceptable use before release.
Nutrition estimate safety	Label estimates, avoid extreme targets, and validate input/output ranges.
Local data loss	Make export/import discoverable and test restore thoroughly.
Schema changes	Use Drift migrations and maintain migration tests.
Unit ambiguity	Store canonical units and explicitly record food reference basis.
Exercise variety	Start with strength-training sets; design extensible fields before adding time/distance modes.
Play Store requirements	Review current privacy policy, Data safety, content rating, and store asset requirements before submission.

Assumptions

- The MVP supports one local profile per installation.
- The first release targets Android; iOS is not a required deliverable.
- Users manually enter food and workout information; automatic tracking is not required.
- The food API is a convenience, not a dependency for core operation.
- The app is a tracking and estimation tool, not a clinical or medical product.

Decisions intentionally deferred

- Final branding, logo, typography, and complete design system.
- Exact starter food catalog contents and external API integration details.
- Whether Android device backup is enabled in addition to manual export/import.
- Exact calorie and macronutrient recommendation policy and guardrails.
- Whether to support non-strength exercise types in the MVP.

Recommended immediate next action

Create the Flutter repository and implement Phase 1: application shell, navigation, theme tokens, Riverpod setup, and the initial Drift database with migrations. Then build onboarding against the real local persistence layer.

End of FitPath project specification